

人工智能原理与应用

詹紫琳

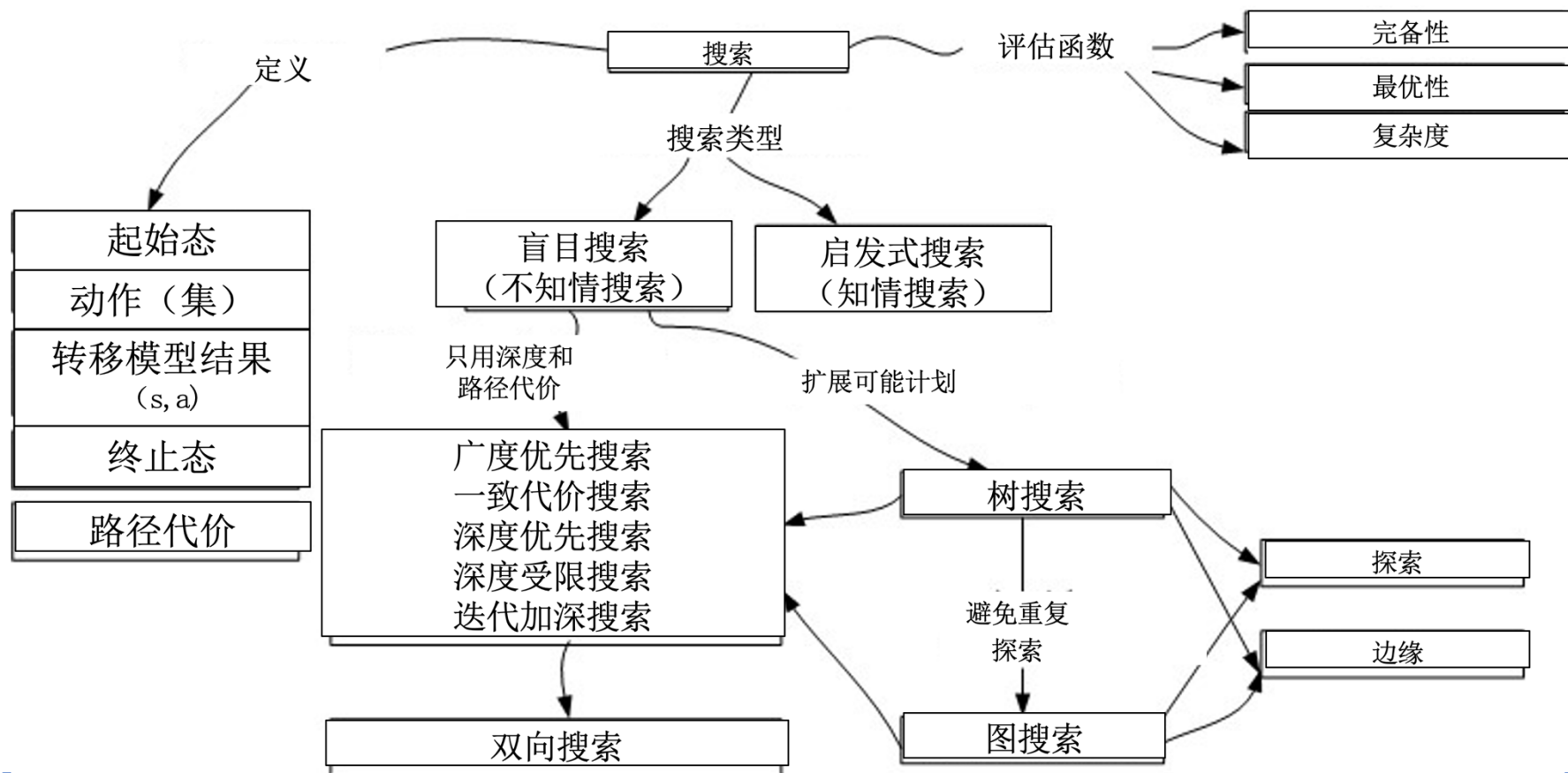
邮箱: zilin.zhan@szu.edu.cn

Spring 2026



深圳大学
SHENZHEN UNIVERSITY

回顾



启发式函数 (Heuristic function)

- 在搜索问题中，每个节点 n 代表一个状态。
- 启发式搜索引入了一个启发式函数 $h(n)$ ，它估计从当前节点到目标的代价/距离。
- $h(n)$ 不要求完全正确，只要比较合理就行。
- 算法用 $h(n)$ 来决定优先扩展哪些节点，从而大大减少无谓的搜索。

假设你在地图上找路：

- 状态 n ：你当前所在的城市。
- 目标：北京。
- 启发式函数 $h(n)$ ：从当前城市到北京的直线距离。
- 直线距离未必等于实际开车距离（真实代价 $g^*(n)$ ），但作为估算，它能帮助你判断哪个方向更可能接近目标。

启发式搜索的典型应用

- 地图导航 / 出行规划：Google Maps、高德地图、Uber 等路径规划
- 机器人路径规划：机器人和无人机在二维/三维环境中寻找最短路径
- 游戏 AI：NPC 路径规划（角色自动走到目标点）
- 自然语言处理 / 机器翻译：选择最有希望的句子候选

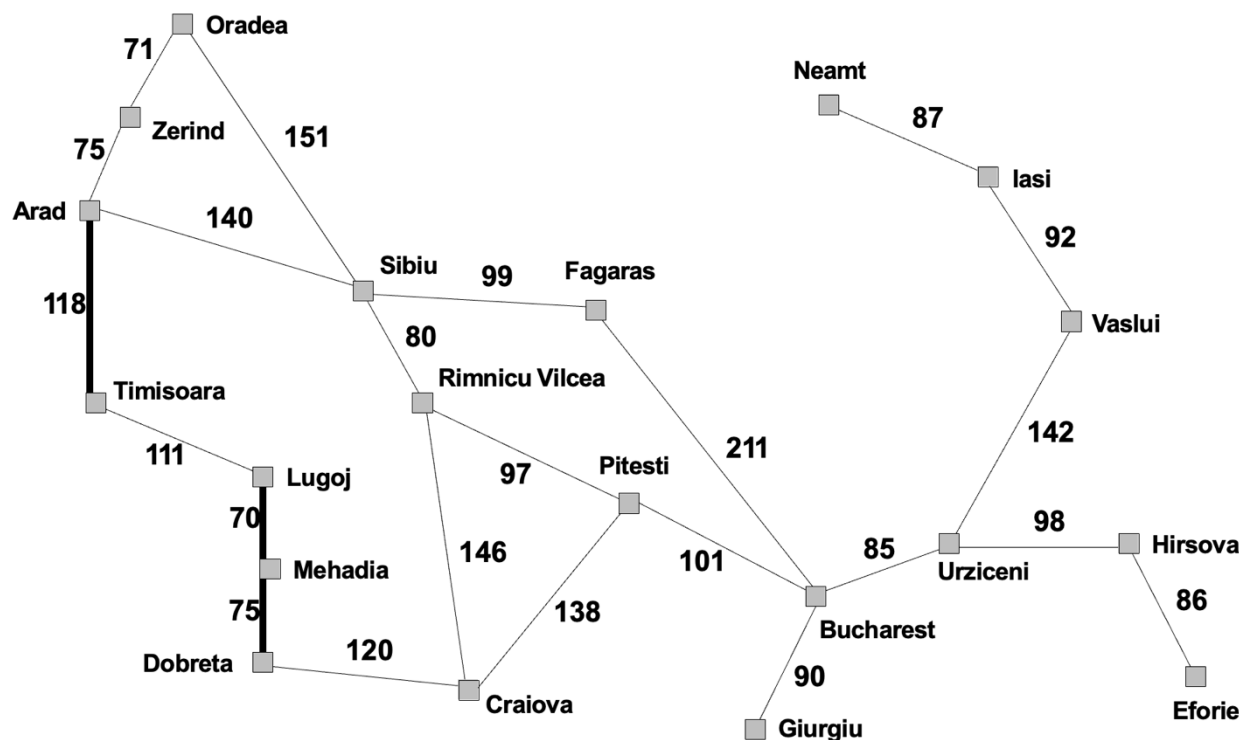
使用启发式函数搜索

- 启发式函数 $h(n)$ 是一个猜测值
- 如果 $h(n_1)$ 具有比 $h(n_2)$ 小的值, 表示我们猜测从 n_1 到达目标的花费更低
- 终止态结点 n , $h(n_{\text{goal}}) = 0$

贪心搜索 (Greedy Best-First Search, GBFS)

- 使用启发式函数 $h(n)$, 估计从当前节点 n 到目标的剩余代价。
- 每次扩展时, 从候选节点中选择 $h(n)$ 最小的那个 (也就是看起来离目标最近的)。贪心——只考虑当下“最优”的结果
- 重复直到找到目标。

欧氏距离 $h(s)$



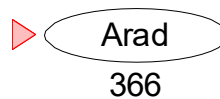
$h(x)$

到达 *Bucharest*
的直线距离

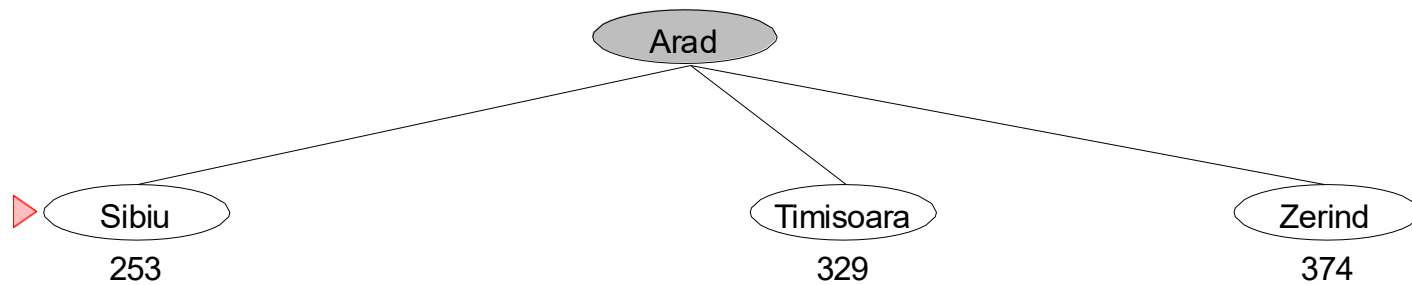
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	241
Oradea	234
Pitesti	380
Rimnicu Vilcea	98
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

假设我们要规划一条从Arad到Bucharest的路线，要求是最大程度地减少起点与目标的距离。

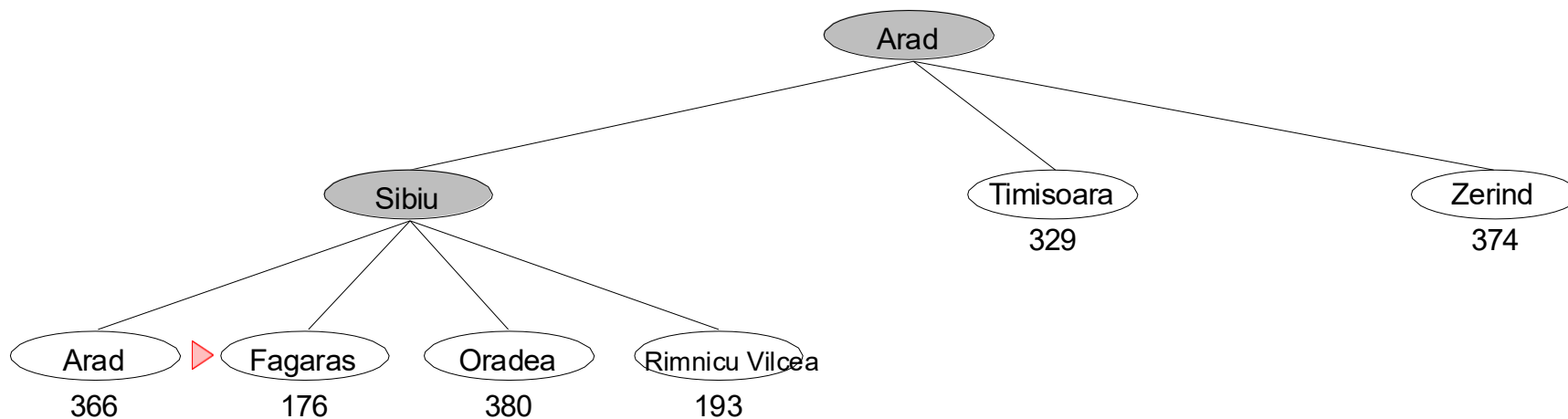
欧氏距离 $h(s)$



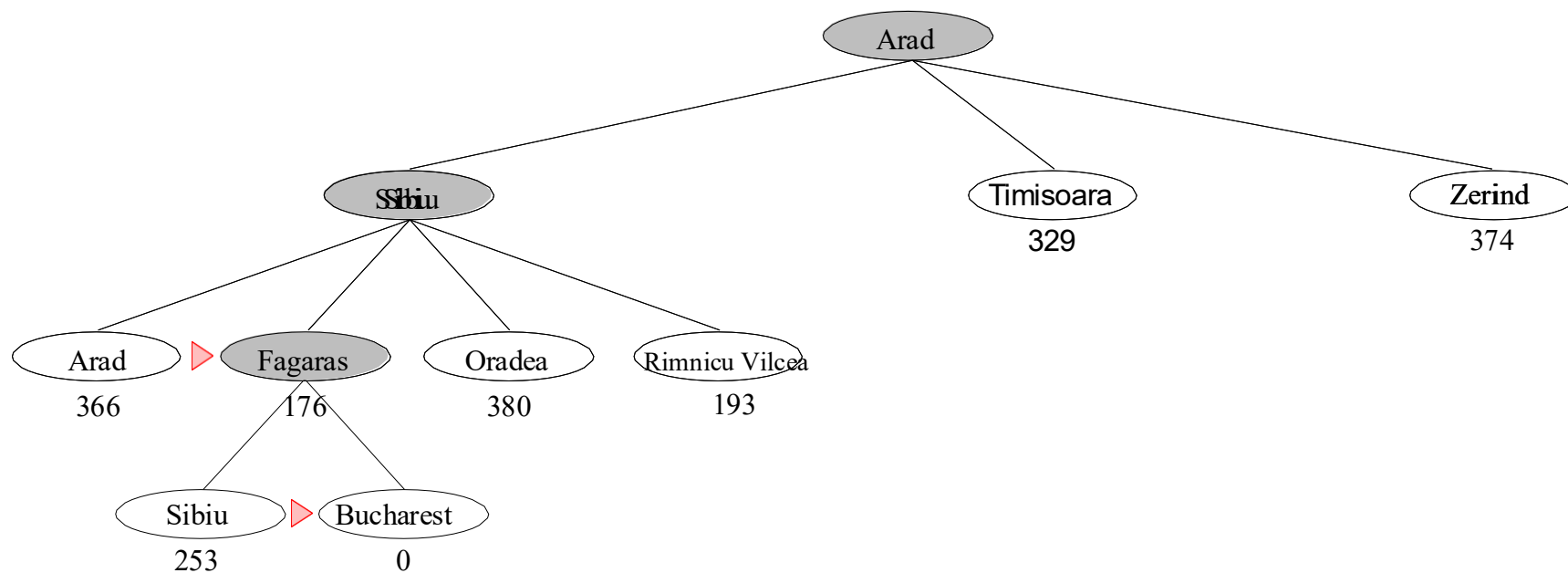
欧氏距离 $h(s)$



欧氏距离 $h(s)$



欧氏距离 $h(s)$



贪心搜索的属性

1. 完备性 (Completeness)

不保证完备

因为贪心搜索可能会掉进死胡同：它总是选择**看起来离目标最近的节点**，但如果路径被障碍物挡住，它可能在一个区域来回徘徊，无法找到解。

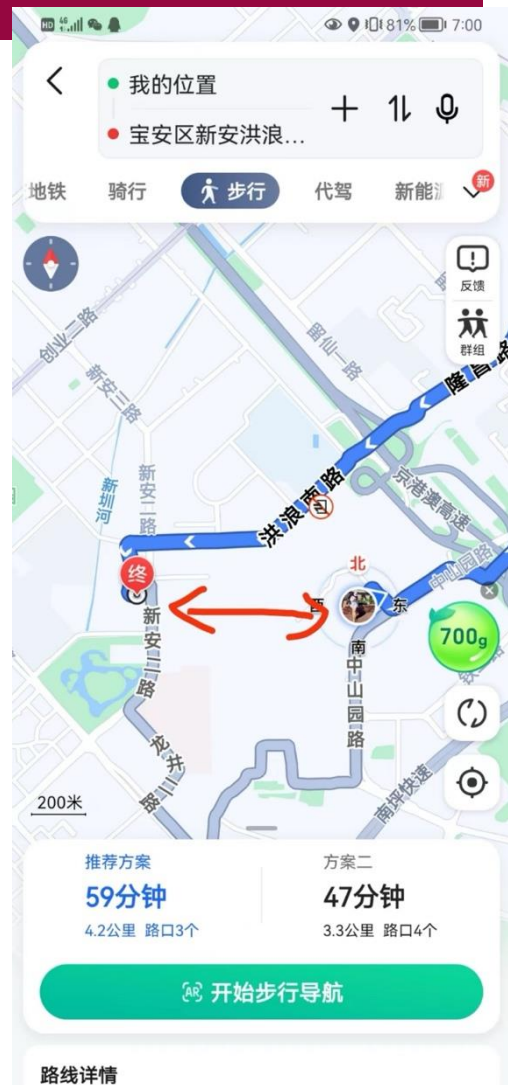
在有限状态空间 + **检查重复状态**的情况下，可以保证完备；否则不一定。

2. 最优性 (Optimality)

不保证最优解

原因：它只依赖启发式函数 $h(n)$ （估计剩余代价），而不考虑从起点到当前节点的真实代价 $g(n)$ 。

所以它可能走了一条“**看起来近，但总成本更大**”的路线。



贪心搜索的属性

3. 时间复杂度 (Time Complexity)

最坏情况下，仍然可能需要扩展指数级的节点：

$$O(b^m),$$

其中 b = 分支因子， m = 最大深度。

实际效率取决于启发式函数的质量。如果 $h(n)$ 很好（接近真实代价），贪心搜索会比盲目搜索（BFS/DFS）快得多。

4. 空间复杂度 (Space Complexity)

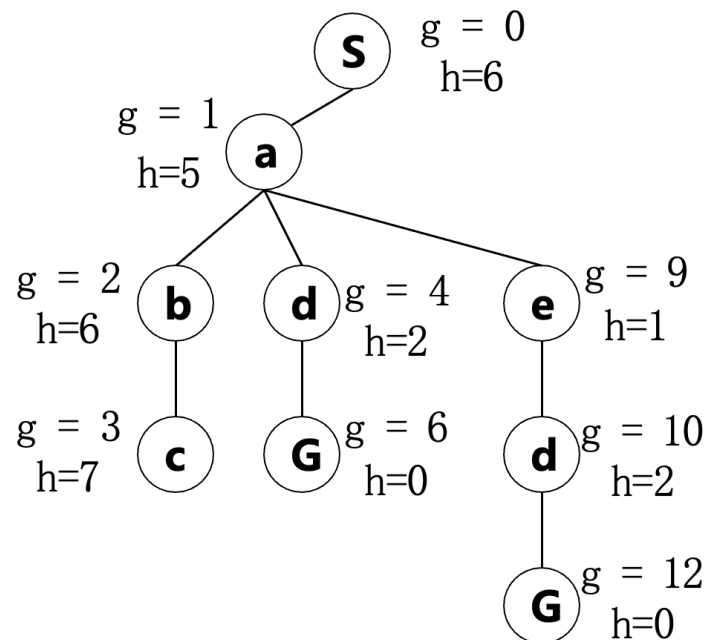
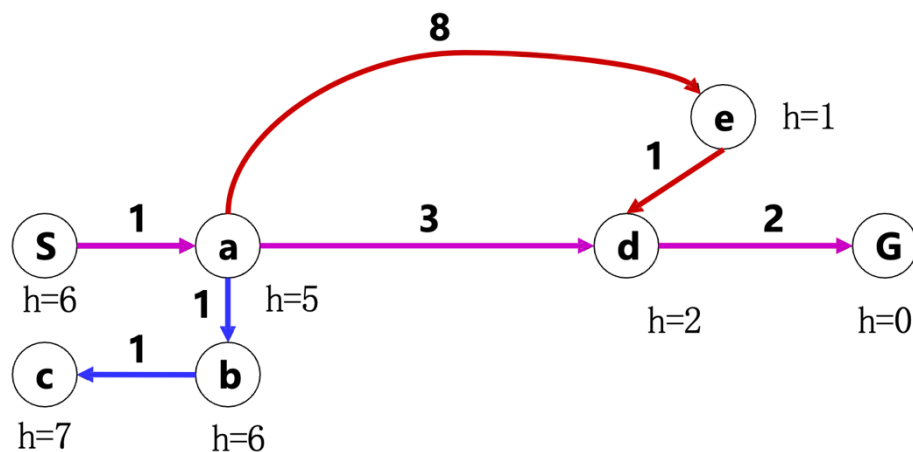
和 UCS 类似，需要维护一个优先队列（按照 $h(n)$ 排序）。

空间复杂度与时间复杂度同量级：

$$O(b^m).$$

A*算法

- 一致代价搜索 按照路径代价或者向后代价 $g(n)$ 顺序
- 最好优先算法 优先按照目标接近度或者前向代价 $h(n)$ 顺序



A*算法

- 最好优先的贪婪搜索算法（GBFS）只考虑 $h(n)$ 的启发式方法忽略了**起点到点n的代价**，若起点到结点n的代价太大，会导致解不是最优。
- 只看“从这里到目标还差多少”，完全忽略了“从起点走到这里已经花了多少代价”。
- GBFS的评价函数：

$$f(n)=h(n)$$

如果某个节点 n ：

到目标的估计代价 $h(n)$ 很小（看起来离目标近），

但实际上从起点到它的**代价 $g(n)$** 很大，

那么贪心搜索仍会优先选择它，因为它只关心 $h(n)$ ，不关心 $g(n)$ 。

A*算法

- 对策：考虑从起点到结点 n 的代价以及从结点 n 的状态到达目标态的代价估计，定义一个评价函数为 $f(n)$:

$$f(n) = g(n) + h(n)$$

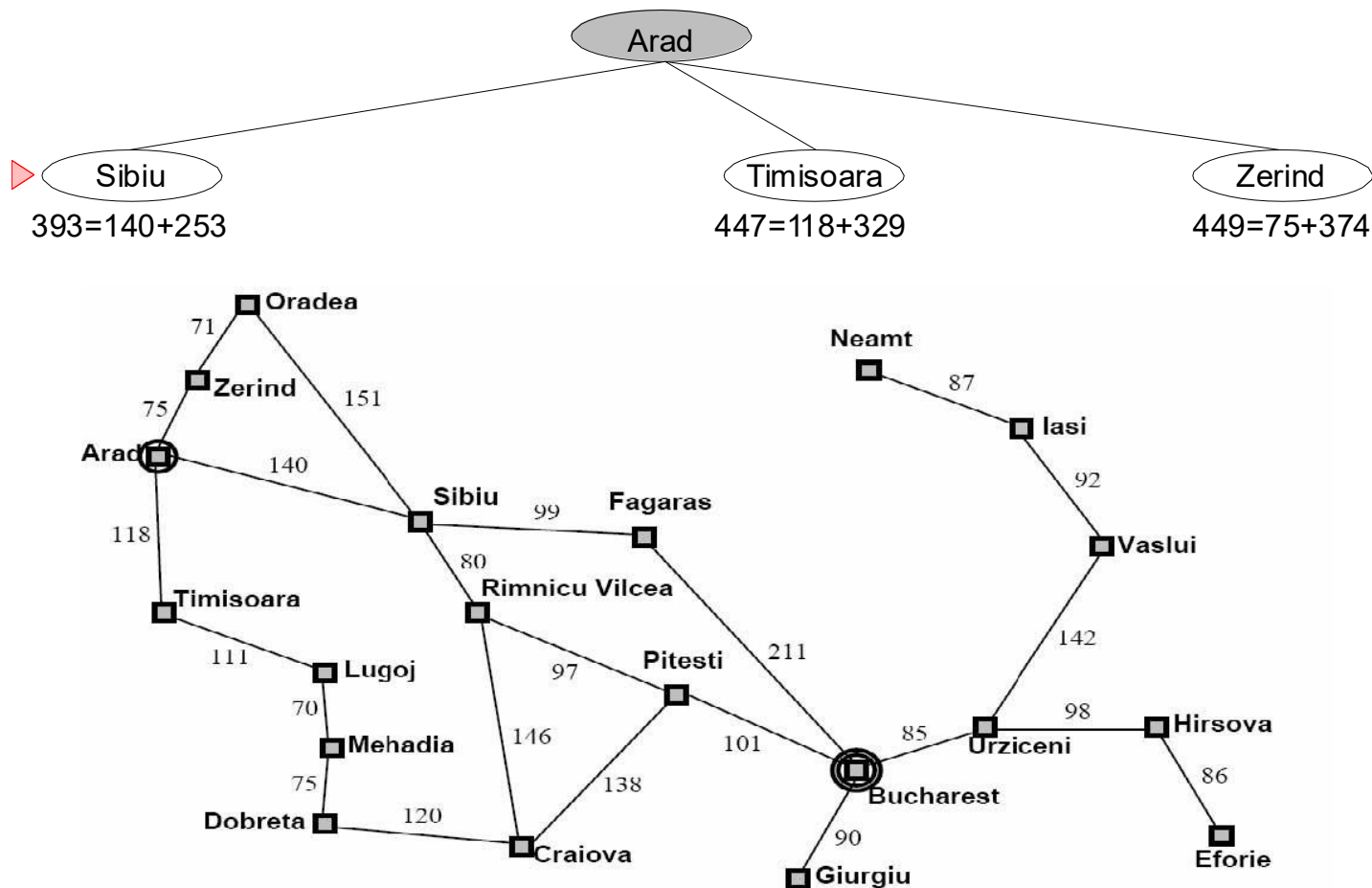
$g(n)$: 从起点到当前结点 n 的实际代价（已经走过的成本）。

$h(n)$: 从结点 n 到目标的启发式估计代价（未来的猜测成本）。

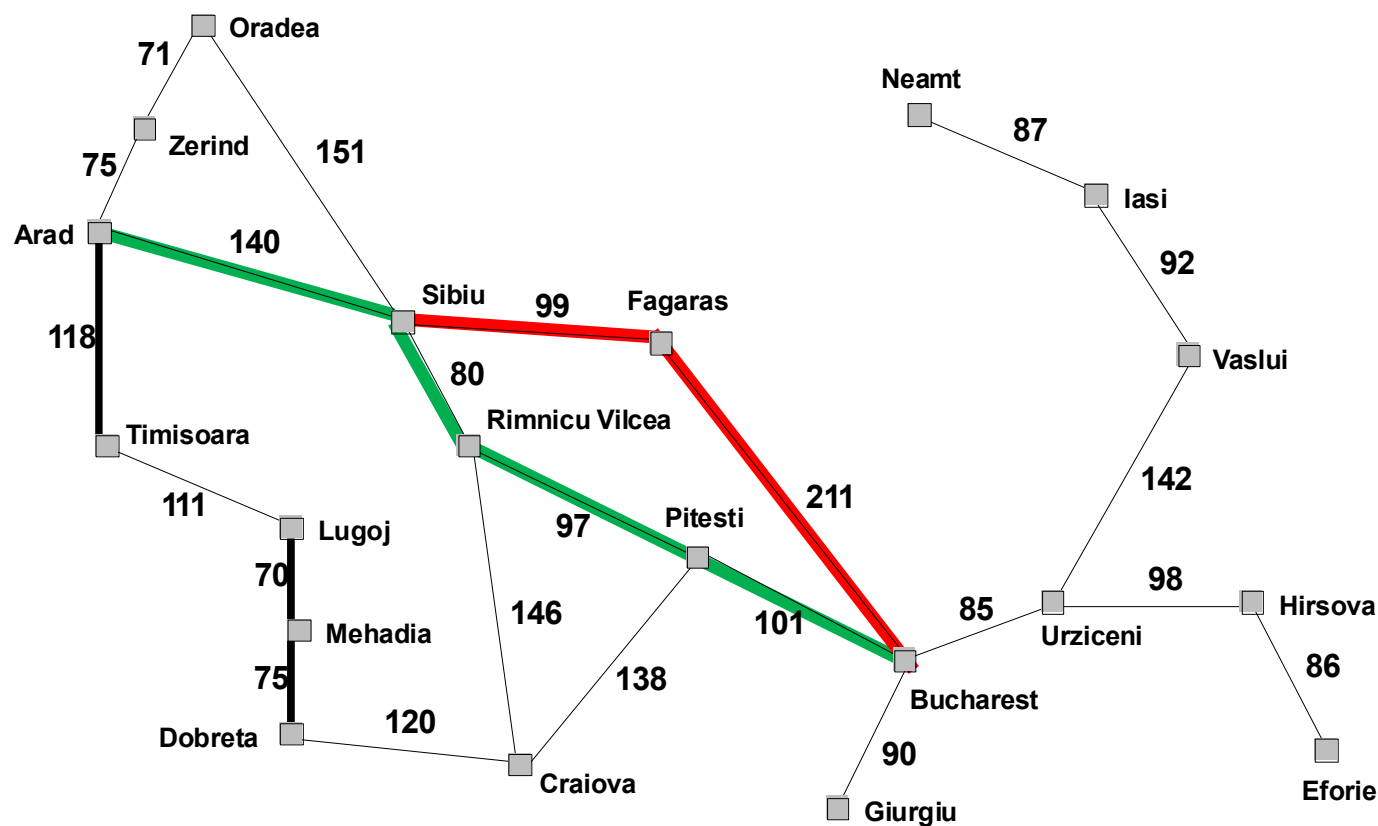
$f(n)$: 总代价估计值（到达目标的完整路径的估算）。

- 动作：始终从边缘上展开具有 **最低 f 值** 的结点

A*算法



A*算法



$h(x)$

到达 *Bucharest*

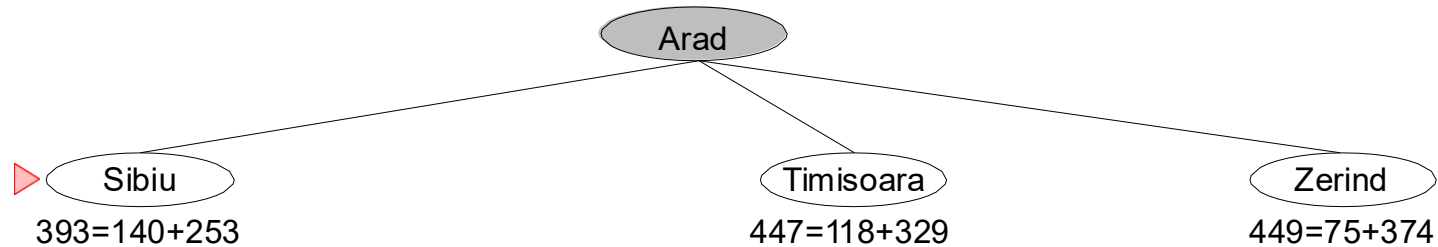
的直线距离

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

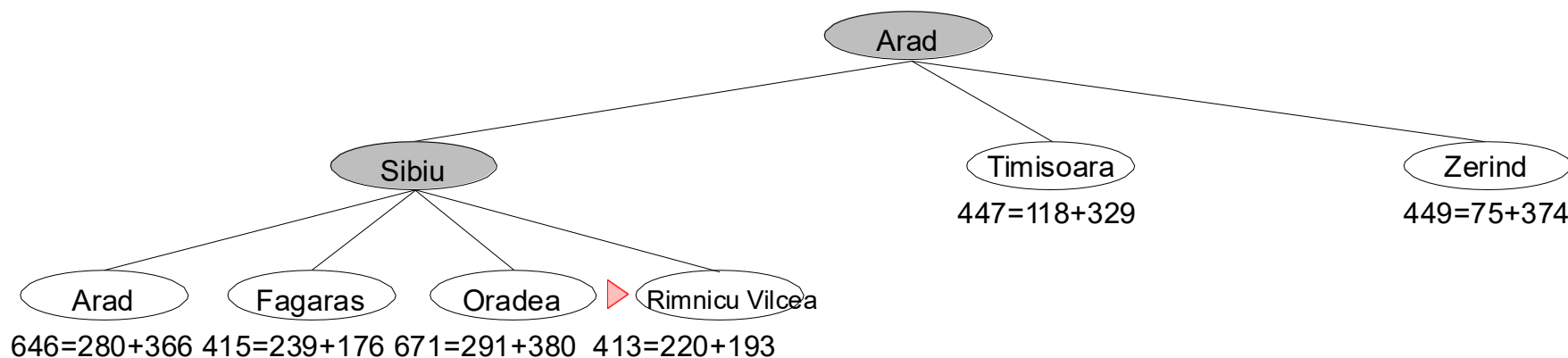
A*算法

▶ Arad
 $366 = 0 + 366$

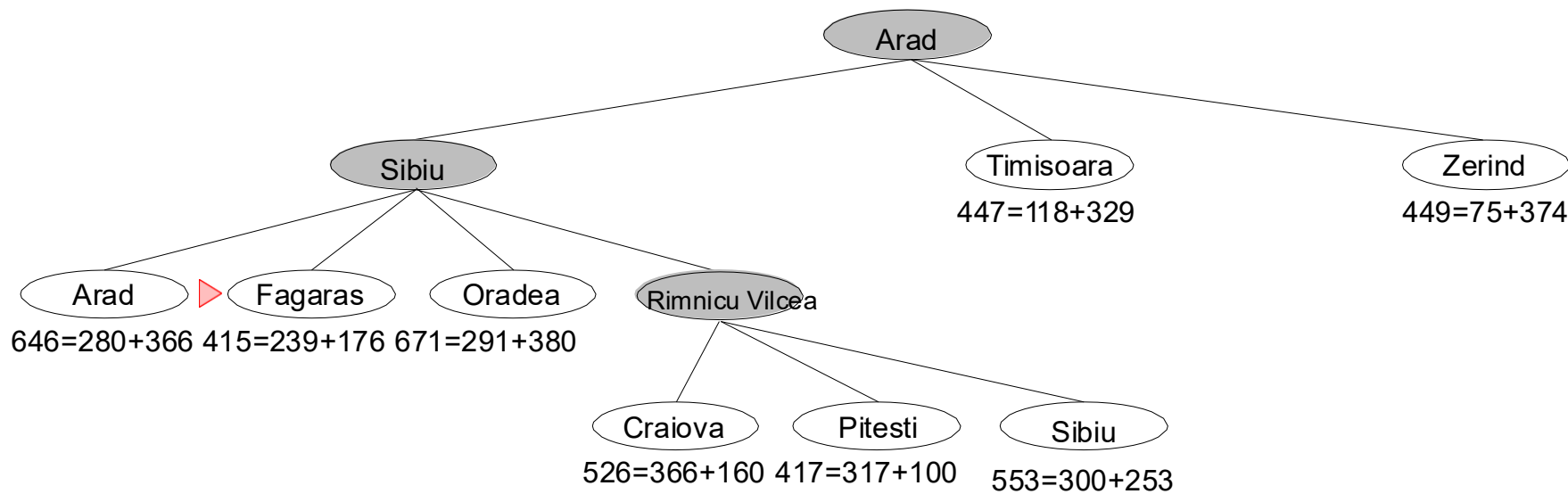
A*算法



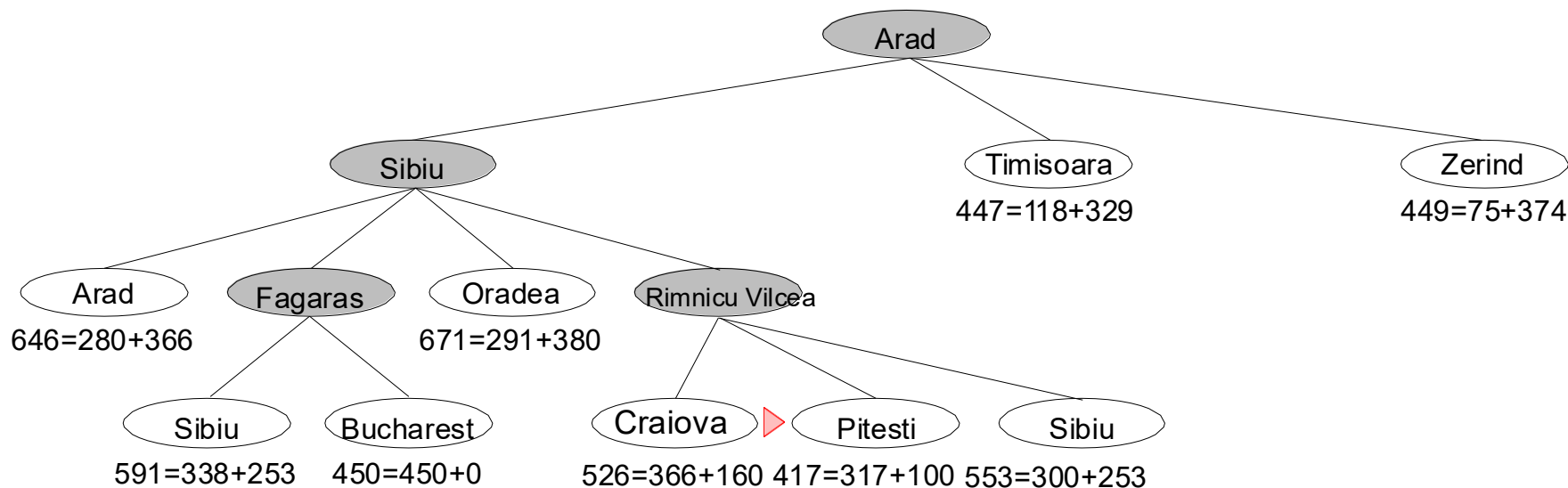
A*算法



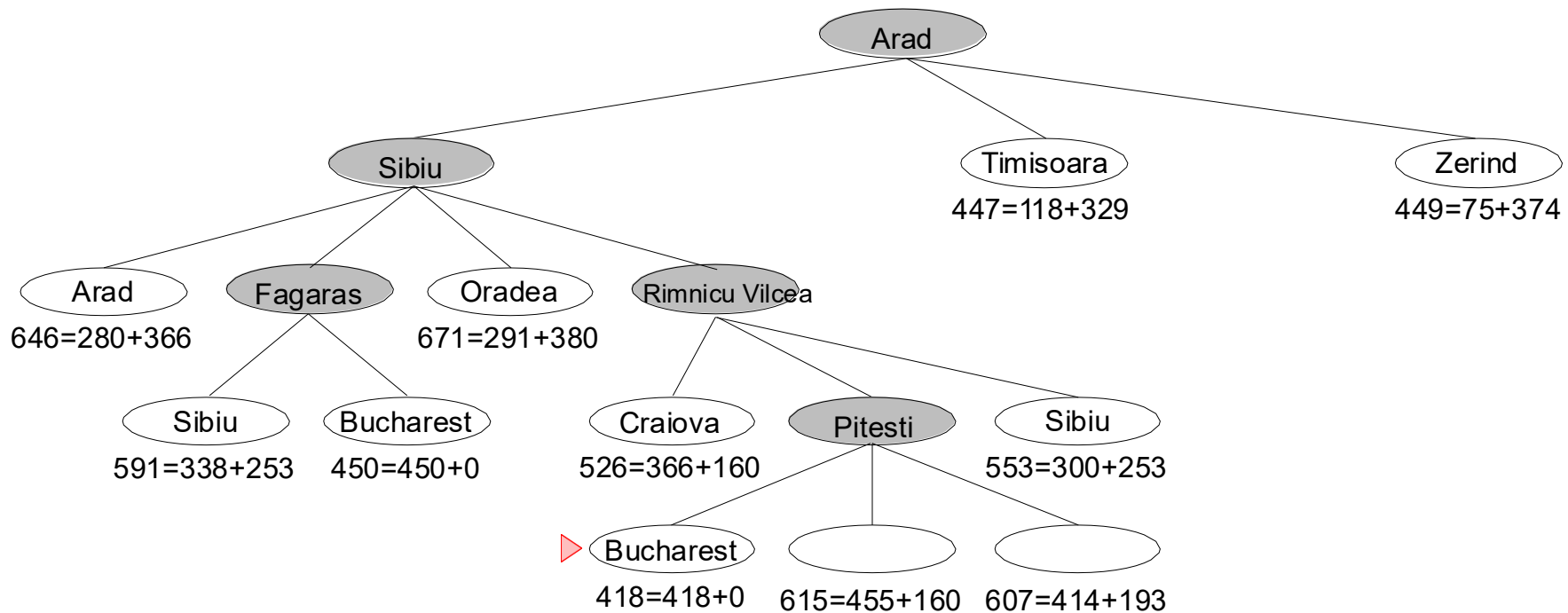
A*算法



A*算法



A*算法

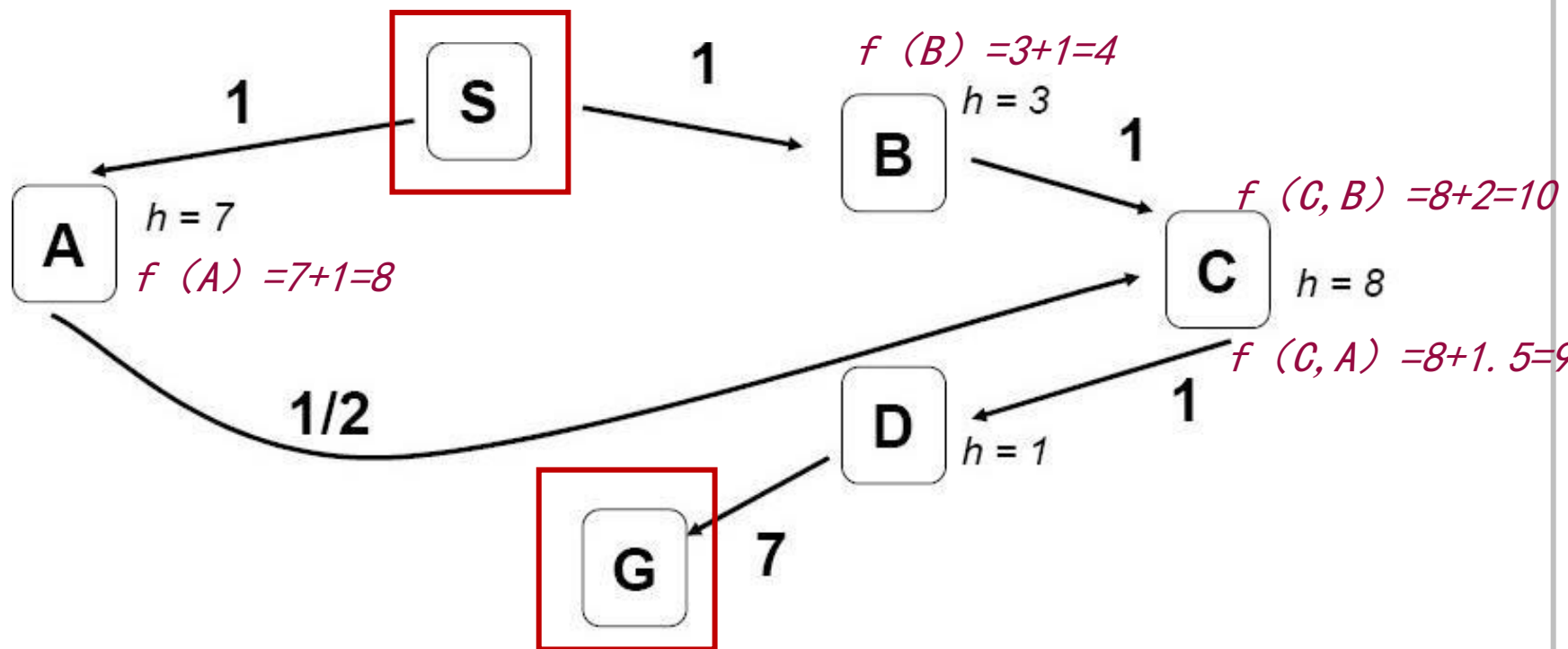


A*算法

伪代码

1. 将起点加入开放列表 OPEN, $f(s)=g(s)+h(s)$, $g(s)=0$
2. 重复以下步骤直到 OPEN 为空:
 - a. 从 OPEN 中取出 $f(n)$ 最小的节点 n
 - b. 若 n 为目标节点, 则路径找到
 - c. 否则:
 - 对 n 的每个后继 n' :
 - 若 n' 不在 OPEN 或新路径更优:
$$g(n') = g(n) + c(n, n')$$
$$f(n') = g(n') + h(n')$$
记录父节点 $\text{predecessor}(n') = n$
3. 若 OPEN 为空但未到目标 $\rightarrow$ 无解

当我们两次访问结点时会发生什么？



路径松弛 (Path Relaxation): 如果A*算法在搜索时通过某个状态发现了一条代价较低的路径, 则应该根据较低的路径代价, 在边缘上更新该状态的顺序。——
—— A*始终在寻找**绝对最好**的路径

A*的性质

1. 完备性 (Completeness)

条件:

每条边代价 $\geq \epsilon > 0$ (即最小正代价)。

搜索空间有限, 或存在解。

结论:

A* 是完备的 <类似UCS>

2. 最优性 (Optimality)

条件:

(1) 启发式函数 $h(n)$ 可采纳 (admissible) :

$$0 \leq h(n) \leq h^*(n)$$

(即 $h(n)$ 永远不高估实际代价 $h^*(n)$) ,

例如:

在地图导航时, 估算的剩余 **$h(n)$ 必须 $\leq$ 真实路程 $h^*(n)$** , 保证最终找到的路径不会被低估 (转而选择一条实际上成本更高的次优路径) 而错过真正的最优解

A*的最优性 (标准证明)

只要启发函数是可采纳的，A* 算法在找到真正的最优目标 G 之前，绝不会“错误地”结束搜索并确定任何次优目标 G_2 。

$f(G_2) > f(G)$ 因为 G_2 是次优的；

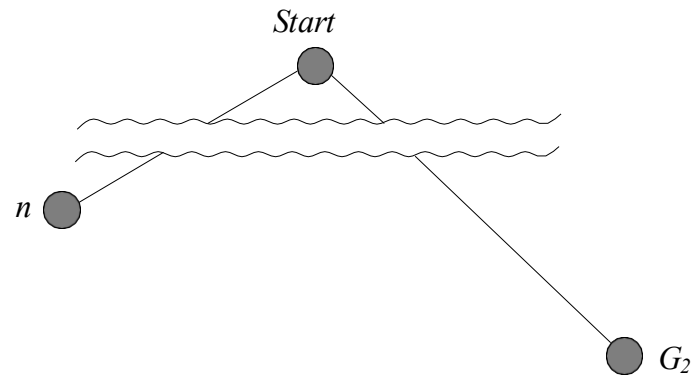
$f(G_2) > g(G)$ 因为 $f(G) = g(G) + h(G)$, 而 $h(G) = 0$;

$g(G) \geq f(n)$ 因为 h 函数是可采纳的，它一定小于或等于实际代价。

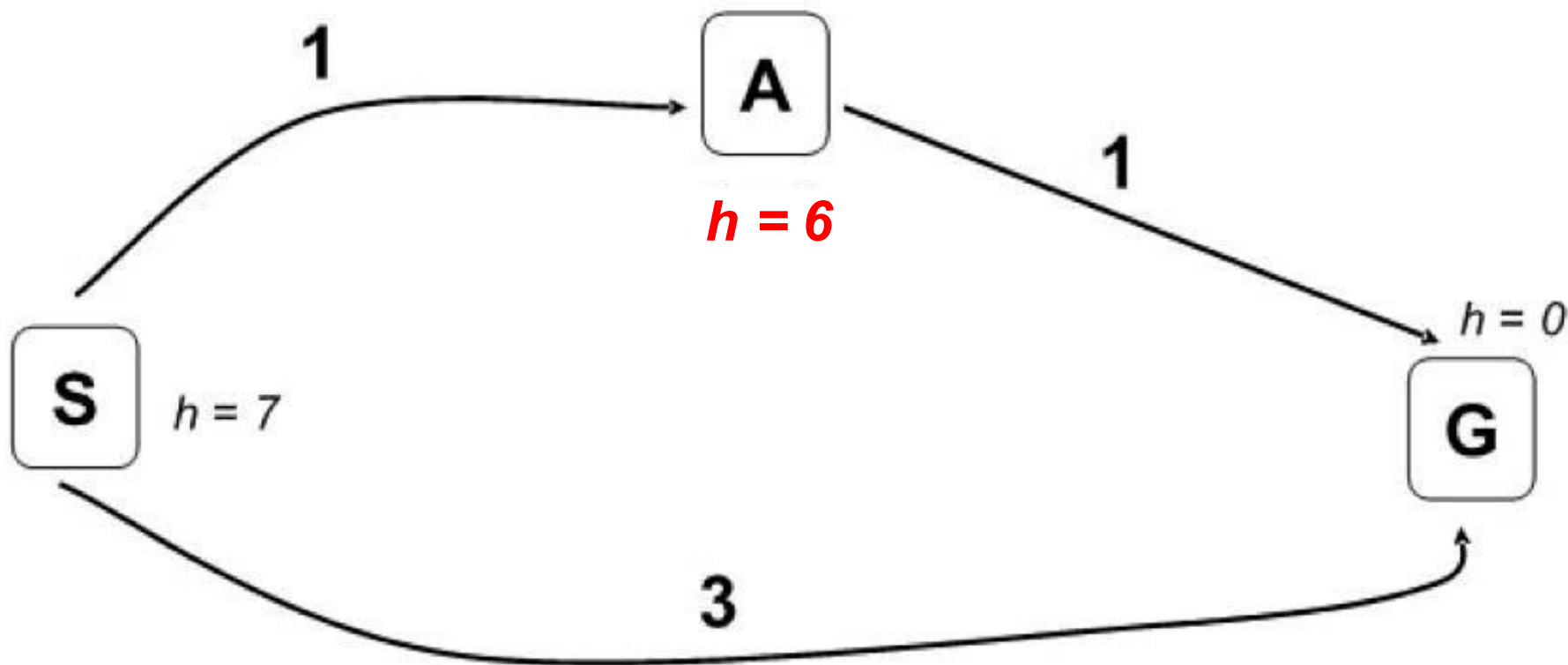
(可采纳性: $g(G) - g(n) \geq h(n)$ $g(G) \geq g(n) + h(n)$
 $g(G) \geq f(n)$)

因此, $f(G_2) > f(n)$;

因为 $f(G) > f(n)$, A* 算法就不会选择 G_2 来扩展了。



A*的最优性 (标准证明)



- ☐ 尝试进行A*搜索，找出了什么问题？
- ☐ 估计目标代价 > 实际目标代价
- ☐ 估计应该少于或最多等于实际代价！

A*的性质

(2) 一致性/单调性 (Consistency/Monotonicity)

启发式函数 $h(n)$ 是一致的, 如果对于任意节点 n 、它的任一后继节点 n' , 以及代价函数 $c(n, a, n')$:

$$h(n) \leq c(n, a, n') + h(n') \quad (\text{三角不等式})$$

其中:

$h(n)$: 当前节点 n 的启发式估计 (到目标的剩余成本预测)

$c(n, a, n')$: 从 n 经过动作 a 到达 n' 的真实代价

$h(n')$: 后继节点 的启发式估计

=> 估计成本是单调递增的, 不会在走几步后突然变得更低。

A*的性质

一致性（或单调性）条件：

$$h(n) \leq c(n, a, n') + h(n')$$

将它转化为与 $f(n)$ 有关的形式：

$$g(n) + h(n) \leq g(n) + c(n, a, n') + h(n')$$

其中，

$$g(n') = g(n) + c(n, a, n')$$

所以：

$$g(n) + h(n) \leq g(n') + h(n')$$

也就是：

$$f(n) \leq f(n') \text{ —— 单调性}$$

A*的性质

举个例子....

你从家 (n) 出发, 要到公司 (Goal)

中途有一个 加油站 (n')

你现在的导航估计: “家 $\rightarrow$ 公司 还要 40 分钟” ($h(n)=40$)

实际上, “家 $\rightarrow$ 加油站” 要 10 分钟 ($c=10$)

“加油站 $\rightarrow$ 公司” 导航估计要 25 分钟 ($h(n')=25$)

那加起来是: $10 + 25 = 35$ 分钟。比一开始估计的 40 分钟更短

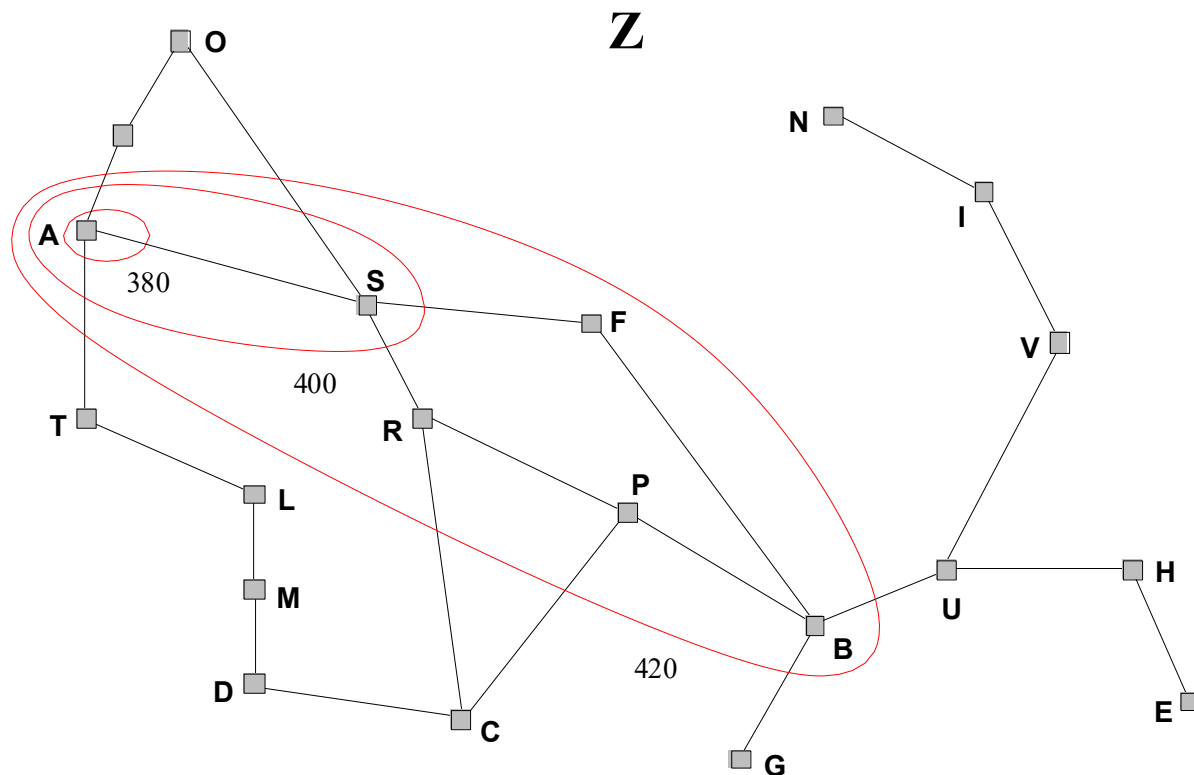
如果反过来——导航说你从家直接到公司只要 30 分钟 (<35), 那就太乐观了, 不一致。

这反映了A*算法的关键性质:

一旦某个节点 n 被算法取出 (即队列中 f 值最小), 我们就已经找到了从起点到 n 的最短路径。后续再扩展的节点, 其 f -值只会更大, 不可能出现“后来有更便宜的路径”这种情况。

A*的最优性 (标准证明)

- 引理: A*以 f 值递增的顺序扩展结点*
- 逐渐添加结点的“ f -等高线”
- 等高线 i 具有所有 $f=f_i$ 的结点, 其中 $f_i < f_{i+1}$



A*的性质

3. 时间复杂度 (Time Complexity)

最坏情况，A* 仍然可能扩展指数级的节点：

$$O(b^d),$$

其中 b = 分支因子， d = 解的深度。

实际效率取决于启发式函数的质量：

启发式越接近真实代价 $h^*(n)$ ，扩展的节点越少

如果 $h(n) = 0$ ，A* 退化为 UCS。

如果 $h(n) = h^*(n)$ ，A* 只扩展最优路径上的节点（理想情况）。

4. 空间复杂度 (Space Complexity)

A* 必须维护 open 表（优先队列）和 closed 表（已访问节点），存储所有生成的节点。

所以空间复杂度和时间复杂度同阶：

$$O(b^d).$$

这也是 A* 的最大缺点之一——空间需求非常大。

A*的性质

- 启发函数估计最短路径的代价
- 好的启发算法可以显著减少算法复杂度（反之，性能高度依赖启发式函数质量） $\Rightarrow$ 可以利用强化学习或深度网络来学习更优秀的启发式函数
- 最好优先的贪婪算法扩展最小的 h
 - 不完备也不保证最优
- A* 算法扩展最小的 $g + h$
 - 完备且最优
 - 也是最有效的

两种启发式算法分别是什么，
并说说各自的完备性和最有性？